

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
 Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1407	Course Title:	COMPILER DESIGN
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	40% of CIA		
CO1 Statement:	Apply lexical analysis for token specification and recognition using LEX tool. (BL3)		
Student Name:	Gautham. B	Reg. No.:	192421318
Section / Batch:	Slot - C	Date:	17/07/26

Code of Conduct

1. Gautham. B / 192421318 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: [Signature]

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	4 / 4	3 / 4	4 / 4	4 / 4	3 / 4	91.25
2	4 / 4	3 / 4	3 / 4	3 / 4	4 / 4	83.75
3	3 / 4	4 / 4	3 / 4	4 / 4	8 / 4	85
4	4 / 4	4 / 4	3 / 4	3 / 4	4 / 4	88.75
5	3 / 4	3 / 4	4 / 4	4 / 4	4 / 4	90
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	87.75
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

N. D. J.

ANALYTICAL PROBLEM SOLVING

1	Trace the processing of the input expression $P = (a * b) + (b * c) * 2$ through all phases of a compiler. a) Identify and list all tokens generated during lexical analysis. b) Construct the syntax tree during the parsing phase. c) Generate the intermediate code for the given expression. d) Apply suitable optimizations to the intermediate code. e) Explain the significance of the final target code prod
2	For each of the following Regular Expressions, construct the language. Write any FIVE valid strings for each Regular Expression. a) $(\epsilon + aa^*)^*$ b) $(a^* + b^*)^* abb$ c) $(a^* b^*)^* aba (a^* + b^*)^*$ d) $(a + ab)^*$ e) $(aba)^* + (a^* b^*)^*$
3	i) Find the number of tokens in the following C statement is <pre>main() { int a=10; char b="abc"; in t c = 30; ch ar d = "xyz"; in /* comment */ t m = 40.5; }</pre> ii) Identify the lexical errors in the following C statement is <pre>void main() { int x=10, y=20; char * a; a= &x; x= 1xab; }</pre>
4	Find a regular expression corresponding to each of the following subsets of $\{0, 1\}^*$: i) The language of all strings that have an even number of 0's. ii) The language of all strings that contain at least one 1. iii) The language of all strings in which both the number of 0's and the number of 1's are odd. iv) The language of all strings that start with 1 and end with 0. v) The language of all strings that do not contain consecutive 1's.

Consider a lexical analyzer for a simplified programming language. The language consists of the following tokens:

1. Keywords: if, else, while, return, int, float
2. Operators: +, -, *, /, =, ==, <, >
3. Separators: (,), {, }, :, ,
4. Identifiers: A sequence of letters followed by a sequence of digits, e.g., var123
5. Integer Literals: A sequence of digits, e.g., 12345
6. Floating-point Literals: A sequence of digits followed by a dot and another sequence of digits, e.g., 123.45

The lexical analyzer needs to tokenize the following input string:

int var123 = 12345; float var456 = 123.45; if (var123 == var456) { return; }

Questions:

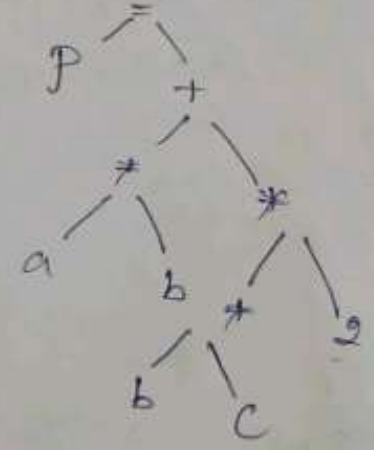
- a) How many tokens are there in the given input string?
- b) Provide a list of tokens categorized by their types (keywords, operators, separators, identifiers, integer literals, floating-point literals).
- c) Assuming a finite state machine (FSM) is used for lexical analysis, estimate the number of state transitions required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.

1) Trace the processing of the expression
 $P = (a+b) + (b+c) * 2$

a) Tokens generated during Lexical Analysis

- Token - Type
- P - Identifier
 - = - Assignment operator
 - (- Left parenthesis
 - a - Identifier
 - + - Addition operator
 - b - Identifier
 -) - Right parenthesis
 - + - Addition operator
 - (- Left parenthesis
 - b - Identifier
 - + - Multiplication operator
 - c - Identifier
 -) - Right parenthesis
 - * - Multiplication operator
 - 2 - Constant
- Total = 15

b) Syntax Tree



c) Intermediate code

- $t_1 = a + b$
- $t_2 = b + c$
- $t_3 = t_2 * 2$
- $t_4 = t_1 + t_3$
- $P = t_4$

d) Optimised Intermediate Code

- $t_1 = b + c$
- $t_2 = t_1 * 2$
- $t_3 = a + b$
- $P = t_3 + t_2$

e) Significance of Final Target Code

- + Produces machine instructions
- + Optimized faster execution
- + Uses CPU registers efficiently

2) Regular Expressions - Any 5 valid strings

a) $(\epsilon + aa^+)^+$

Possible strings:

ϵ
a
aa
aaa
aaaa

b) $(a^+ + b^+)^+ abb$

Ends with abb

Strings:

abb
aabb
babb
aaabb
bbabb

d) $(a + ab)^+$

Strings:

ϵ
a
ab
aaba
babab

c) $(a + b)^+ aba (a^+ + b^+)^+$

Contains aba

Strings:

aba
abaa
abab
aaba
babab

e) $(abab)^+ + (a^+b^+)^+$

Strings:

ϵ
abab
abababab
aaabbbb
aab

3) 1) Number of tokens

Program - C

main()

{

int a = 10;

char b = "abc";

int c = 30;

char d = "xyz";

int m = 40.5;

}

Tokens = 31

Tokens

Statement

Tokens

main()

4

{

1

int a = 10;

5

char b = "abc";

5

int c = 30;

5

char d = "xyz";

5

int m = 40.5;

5

}

1

ii) Lexical Errors

void main()

{ int x=10, y=20;

char *a;

a = &x;

x = 1xab;

}

1. 1xab is an invalid hexadecimal constant

2. Assigning &x (integer address) to char * a is a type error.

4) Regular Expressions

i) Even number of 0's

$1 + (01^*01^*)^*$

ii) At least one 1

$(0+1)^*1(0+1)^*$

iii) Odd number of 0's and odd number of 1's

$(00)^*0(11)^*1(00)^*1(11)^*1)^*$

iv) Starts with 1 and ends with 0.

$1(0+1)^*0$

v) No consecutive 1's.

$(0+10)^*(\epsilon+1)$

5) Lexical Analyzer

Input -

int Var123 = 12345;

float Var456 = 123.45;

{ Var123 == Var456 }

{

return;

}

a) Number of tokens

Token	Type
Print	Keyword
Var123	Identifier
=	Operator
12345	Integer Literal
;	Separator
Float	Keyword
Var456	Identifier
=	Operator
12345	Floating Literal
;	Separator
if	Keyword
{	Separator
Var123	Identifier
==	Operator
Var456	Identifier
}	Separator
{	Separator
return	Keyword
:	Separator
}	Separator
Total = 20	

b) Categorized Tokens

Keywords
 + int + float + if
 + return
 Count = 4

Operators
 * =
 * =
 * == Count = 3

Separators
 + ; + { + }
 + { + ; + }
 Count = 7

Identifiers
 + Var123 + Var456
 + Var123 + Var456
 Count = 4

Integer Literals
 + 12345 Count = 1

c) Estimated FSM state Transitions

Keywords (4) - 18
 Identifiers (4) - 24
 Operators (3) - 4
 Separators (7) - 7
 Integer Literal - 3
 Floating Literal - 7